Assignment #2 Image Enhancement

測量115 F64111134 施淳譯

1. Histogram equalization:
   Goal: Map the histogram to another one whose probability density function (PDF) is a constant, by using the cumulative distribution function (CDF) to remap the image intensity values.

```python
def hist_equalize_gray(img):
    hist = np.bincount(img.flatten(), minlength=256)
    pdf = hist / img.size
    cdf = np.cumsum(pdf)
    cdf_norm = (cdf - cdf.min()) / (cdf.max() - cdf.min()) # Normalize to [0,1]
    mapping = np.round(255 * cdf_norm).astype(np.uint8) # Scale to [0,255]
    return mapping[img]
```

Figure 1. Code for histogram equalization

Steps:

a. Calculate the histogram of the image between 0-255, (x-axis represents intensity, y-axis represents pixel counts).
b. Convert the histogram into the PDF (x-axis:intensity, y-axis:probability).
c. Compute the CDF (x-axis:intensity, y-axis:cumulative probability).
d. Normalize the CDF to the range [0-1].
e. Use the normalized CDF as a mapping function to remap each intensity value to the new range [0, 255].

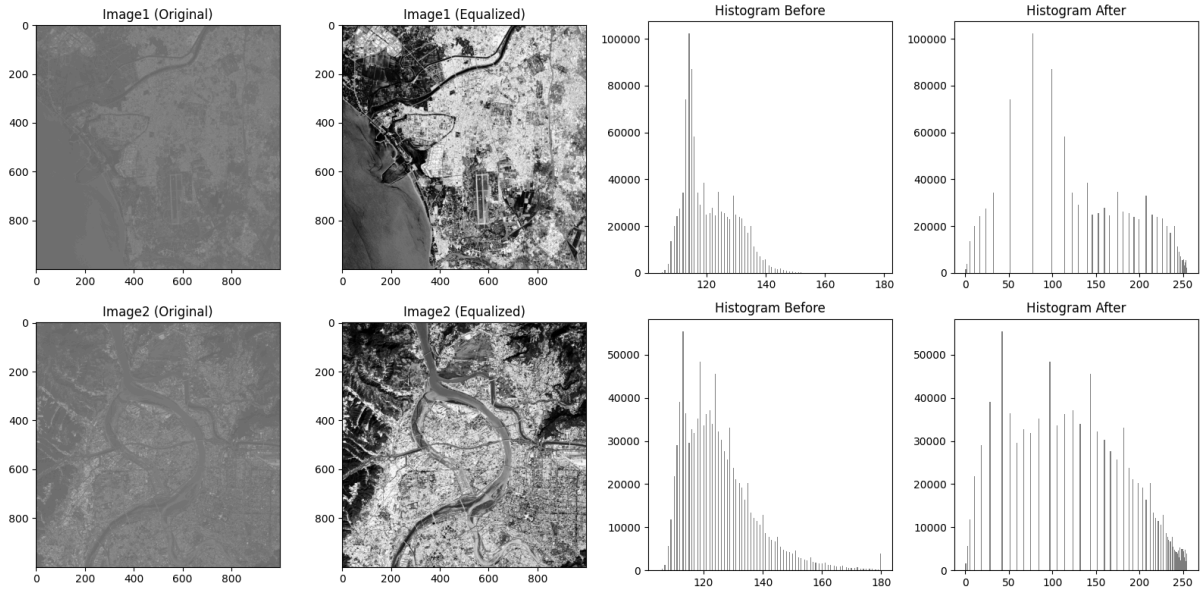


Figure 2. Histogram equalization result

2. The reason why discrete histogram equalization does not yield a flat (平坦的) histogram is that gray-level intensities are finite and must be represented as integers. This quantization makes the values discrete rather than continuous, so the transformation function is replaced by a summation instead of an integral. As a result, the CDF becomes a stair like (階梯型的) function, and multiple input intensity levels may map to the same output level (many-to-one mapping).

Therefore, the discrete histogram equalization cannot produce a perfectly flat histogram, but only an approximately uniform one.

3. Histogram matching:
   Goal: Adjust the shape of the input image's histogram so that it resembles that of a particular reference image.

```python
def histogram_match_gray(src, ref):
    hist_src, _ = np.histogram(src.flatten(), bins=256, range=(0, 256))
    hist_ref, _ = np.histogram(ref.flatten(), bins=256, range=(0, 256))
    cdf_src = np.cumsum(hist_src).astype(np.float64)
    cdf_ref = np.cumsum(hist_ref).astype(np.float64)
    cdf_src = cdf_src / cdf_src[-1] # Normalize to [0,1]
    cdf_ref = cdf_ref / cdf_ref[-1]
    mapping = np.interp(cdf_src, cdf_ref, np.arange(256)) # Map src to ref
    mapping = np.clip(np.round(mapping), 0, 255).astype(np.uint8) # Ensure valid range
    return mapping[src]
```

Figure 3. Code for histogram matching

Steps:

a. Calculate the histograms of the source and reference images.
b. Compute the CDF of both images.
c. Normalize the CDF to the range [0,1].
d. Use interpolation to find the mapping function that transforms each intensity level in the source image to the corresponding value in the output range [0, 255]. The reference image's CDF already defines the relationship between cumulative probability and gray level, so interpolation is used to match the source CDF to that mapping.
e. Ensure that the mapped values stay within the valid range [0, 255] and convert them to integer type.

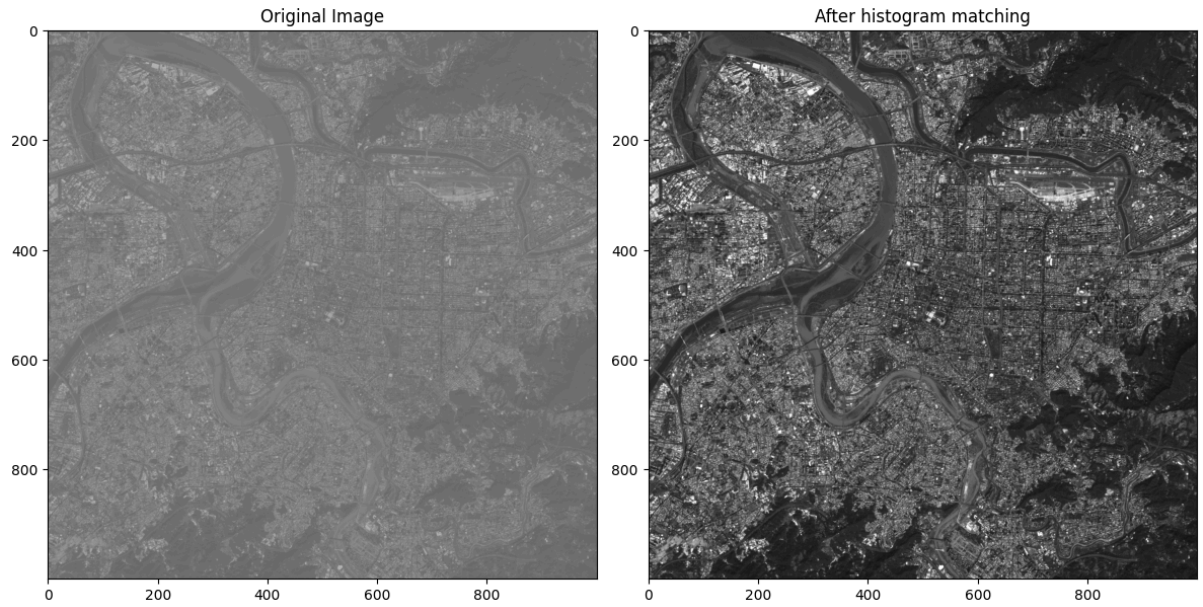


Figure 4. Histogram matching result

4. Average and Sobel filter

   Goal: Implement the Average and Sobel filter using 2D convolution

```python
def make_filter(ftype, direction=None):
    if ftype == 'average':
        kernel = np.ones((3, 3)) / 9
    elif ftype == 'sobel':
        if direction == 'vertical':
            kernel = np.array([[-1,  0,  1],
                               [-2,  0,  2],
                               [-1,  0,  1]])
        elif direction == 'horizontal':
            kernel = np.array([[-1, -2, -1],
                               [ 0,  0,  0],
                               [ 1,  2,  1]])
    return kernel

def Conv2D(image, kernel):
    (h, w) = image.shape
    (fh, fw) = kernel.shape
    pad_h, pad_w = fh // 2, fw // 2
    padded_image = np.pad(image, ((pad_h, pad_h), (pad_w, pad_w)), mode='edge')
    output = np.zeros_like(image, dtype=float)
    for i in range(h):
        for j in range(w):
            region = padded_image[i:i+fh, j:j+fw]
            output[i, j] = np.sum(region * kernel)
    output = np.abs(output)
    output = np.clip(output, 0, 255)
    return output.astype(np.uint8)
```

Figure 5. Code for creating Average/Sobel filters and performing 2D convolution

Steps:

a. Create 3*3 filters for both the Average and Sobel operations. The average filter is a 3*3 matrix where every element is 1/9, which smooths the image by reducing local variations. The Sobel filter has two orientations:vertical and horizontal. The vertical Sobel filter emphasizes vertical edges. Its first and third columns are negative and positive respectively, with weights [1, 2, 1]. The horizontal Sobel filter emphasizes the horizontal edges. Its first and third rows are negative and positive respectively, with a similar weighting pattern [1, 2, 1].

b. Perform convolution between the image and the selected filter.

c. Get the image and filter shape to determine the required padding size.

d. Pad the image by half the filter size. Padding ensures that every pixel in the original image can serve as the center pixel during convolution. The `mode=edge` parameter copies the border pixel values outward, preventing artificial black borders.

e. Convolve the padded image with the filter and store the sum of element-wise products as the output pixel value.

f. Take the absolute value of the gradient, clip the result to the range [0, 255], and convert it to integer type for display

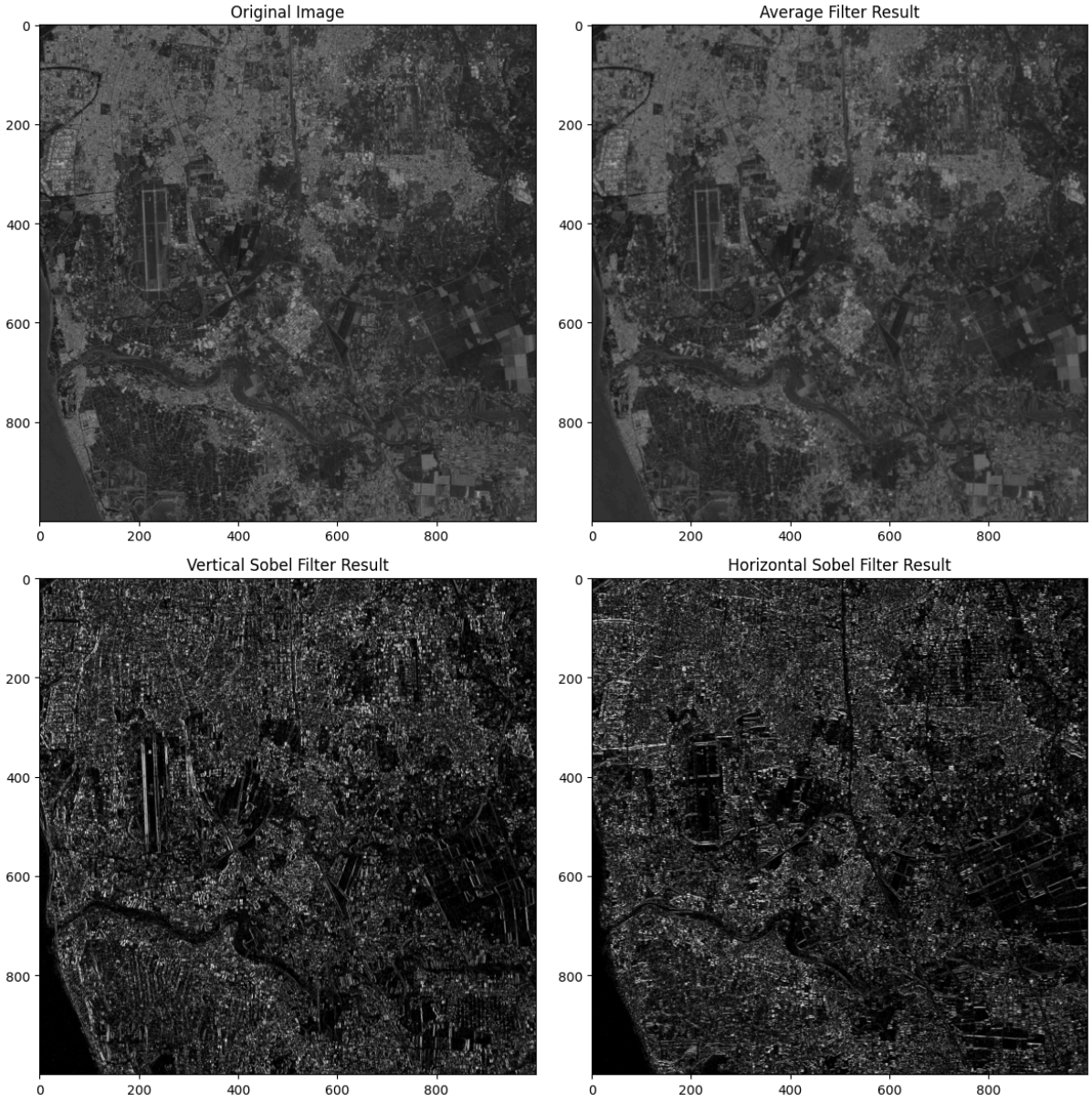


Figure 6. Average/Sobel filtering result